

Guia de smoke test

CloudOrders Payment Worker - Azure Service Bus

Objetivo: validar manualmente o fluxo completo **POST /orders -> OrderCreated -> subscription payment -> Payment Worker -> pagamento simulado -> complete**.

Antes de começar

- Tenha .NET 10, Docker Desktop, Azure CLI e acesso a um namespace Azure Service Bus Standard.
- Tenha PostgreSQL local configurado para a API, com migration aplicada.
- Nunca grave ou envie a connection string do Service Bus em arquivos versionados, chat ou screenshots.
- Se o namespace e o topic **order-events** já existem, execute apenas o passo da subscription payment.

1. Criar ou validar a subscription payment

No PowerShell, substitua os valores de resource group e namespace. A remoção da regra \$Default é essencial: sem isso a subscription receberia todas as mensagens do topic.

```
$resourceGroup = "rg-cloudorders-dev"
$namespace = "seu-namespace-servicebus"
$topic = "order-events"
$paymentSubscription = "payment"

az servicebus topic subscription create `
  --resource-group $resourceGroup --namespace-name $namespace `
  --topic-name $topic --name $paymentSubscription
az servicebus topic subscription rule delete `
  --resource-group $resourceGroup --namespace-name $namespace `
  --topic-name $topic --subscription-name $paymentSubscription `
  --name '$Default'
az servicebus topic subscription rule create `
  --resource-group $resourceGroup --namespace-name $namespace `
  --topic-name $topic --subscription-name $paymentSubscription `
  --name order-created-only `
  --filter-sql-expression "sys.Label = 'CloudOrders.Orders.OrderCreated'"
```

Confira a regra criada:

```
az servicebus topic subscription rule list `
  --resource-group $resourceGroup --namespace-name $namespace `
  --topic-name $topic --subscription-name $paymentSubscription `
  --output table
```

Resultado esperado: somente a regra **order-created-only**, filtrando `sys.Label = 'CloudOrders.Orders.OrderCreated'`.

2. Configurar os segredos locais

Use uma policy SAS com permissões Send e Listen. Guarde a connection string somente nos User Secrets:

```
dotnet user-secrets set --project src/CloudOrders.Api `
  "ConnectionStrings:ServiceBus" "<connection-string>"
dotnet user-secrets set --project src/CloudOrders.Payment.Worker `
  "ConnectionStrings:ServiceBus" "<connection-string>"
```

A API também precisa de `ConnectionStrings:Postgres`. Caso ainda não tenha um banco local em execução, siga a seção Local development do README antes de continuar.

3. Iniciar os processos

Abra dois terminais na raiz do repositório.

Terminal	Comando
API	<code>dotnet run --project src/CloudOrders.Api --launch-profile http</code>
Payment Worker	<code>dotnet run --project src/CloudOrders.Payment.Worker -- --environment Development</code>

No log do worker, confirme a inicialização para topic **order-events** e subscription **payment** antes de enviar o pedido.

4. Criar um pedido

```
$order = Invoke-RestMethod -Method Post `
  -Uri http://localhost:5049/orders `
  -ContentType application/json `
  -Body '{"customerId":"customer-123"}'
$order
```

Resultado esperado na API: resposta **201 Created** com id, customerId, status Pending e createdAtUtc.

5. Validar o processamento

- No Payment Worker: um log de pagamento processado contendo OrderId, MessageId, CorrelationId e a referência simulated-payment-... .
- Em seguida: um log de mensagem concluída para o mesmo OrderId, MessageId e CorrelationId.
- O CorrelationId deve ser o id do pedido em formato GUID.
- No Azure Portal, abra Service Bus Explorer ou métricas de order-events / payment e confirme que não há mensagem ativa após a conclusão.

Critérios de aprovação

Verificação	Esperado
API	POST /orders retorna 201 Created.
Filtro	payment recebe somente subject CloudOrders.Orders.OrderCreated.
Worker	Processa Pending, registra os três identificadores e completa a mensagem.
Subscription	A mensagem deixa de ficar ativa após o complete.

6. Teste opcional de isolamento

Este passo confirma que a regra payment não captura o probe de transporte. Em um terceiro terminal, inicie o worker de probe:

```
dotnet run --project src/CloudOrders.Messaging.Worker -- --environment Development
```

Depois publique o probe:

```
Invoke-RestMethod -Method Post -Uri http://localhost:5049/messaging/probe
```

- O Messaging Worker deve registrar e concluir o probe.
- O Payment Worker não deve registrar esse probe.

Como interpretar falhas

Sintoma	Ação sugerida
Worker não inicia	Confira ConnectionStrings:ServiceBus nos User Secrets e permissões Listen da policy SAS.
POST /orders falha	Confira PostgreSQL, ConnectionStrings:Postgres e a migration aplicada. A publicação ocorre depois da persistência.
Mensagem não chega ao worker	Confira topic, subscription payment, remoção de \$Default e regra order-created-only.
Mensagem continua ativa	Confira logs de exceção do worker. Falhas de subject, content type, JSON ou status não suportado não executam complete.
Status inesperado	O JSON pode ser desserializado, mas o simulador atual aceita apenas Pending. Isso é regra de negócio, não falha de transporte.

Escopo deste smoke test: validar publicação, filtro, consumo e completion. Não introduza nem espere idempotência, retry customizado, DLQ customizada, gateway de pagamento ou persistência de pagamentos.